



Bug Whisperer — AI Debugging Detective

iQOO Hackathon 2026 | Pune City Battle

Slide 1: The Problem

Every Developer Knows This Feeling

It's 2 AM. Your app just crashed. You're staring at a wall of stack trace text — cryptic file paths, line numbers, function names scattered across multiple files. You have no idea **where** to start looking.

The Pain Points:

- Stack traces reference 5-10 files across your codebase
- The error location is rarely the **root cause**
- Senior engineers spend hours tracing causal chains
- Junior developers are completely lost
- Existing AI tools only look at **one file at a time**

The Cost:

Slide 2: Our Solution

Bug Whisperer — The Debugging Detective

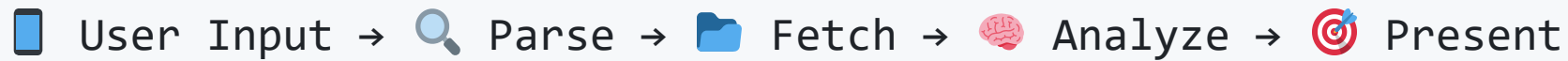
One-liner: Paste a stack trace and a repo link. Bug Whisperer investigates across your **whole codebase** like a detective — pulls the relevant files, reconstructs the causal chain, names the root cause, and proposes the fix.

What Makes Us Different:

Traditional AI Tools	Bug Whisperer
Analyzes ONE file	Analyzes ENTIRE codebase
Shows where error occurred	Traces ROOT CAUSE
Text output	Interactive CASE BOARD
Generic suggestions	Detective-style reasoning

Slide 3: How It Works

The Investigation Pipeline



Step 1: Input

- User pastes a GitHub repo URL + stack trace
- Or photographs a stack trace with phone camera

Step 2: Parse

- Extracts `{file, line, function}` for each stack frame
- Supports Python, JavaScript, Node.js, and more

Step 3: Fetch

- Automatically fetches referenced files from GitHub API

Slide 4: Key Features

Core Features

1. **Multi-File Analysis** — Reasons across your entire codebase, not just one file
2. **Root Cause Detection** — Identifies the TRUE cause, not just where the error surfaced
3. **Role Classification** — Labels each file as Crime Scene, Prime Suspect, Accomplice, or Red Herring
4. **Confidence Rating** — Honest about uncertainty (High/Medium/Low)
5. **Copy-to-Clipboard Fix** — Ready-to-use code fix

Mobile-First Features

6. **Camera Input** — Photograph stack traces from phone camera
7. **PWA Installable** — Install as native app on any phone

Slide 5: UI Design — The Case Board

Screen 1: Input

 Bug Whisperer

[Try Demo: Python | JavaScript]

 Repository URL

 Stack Trace

 Camera

Traceback (most recent
File "app.py", line 15
...

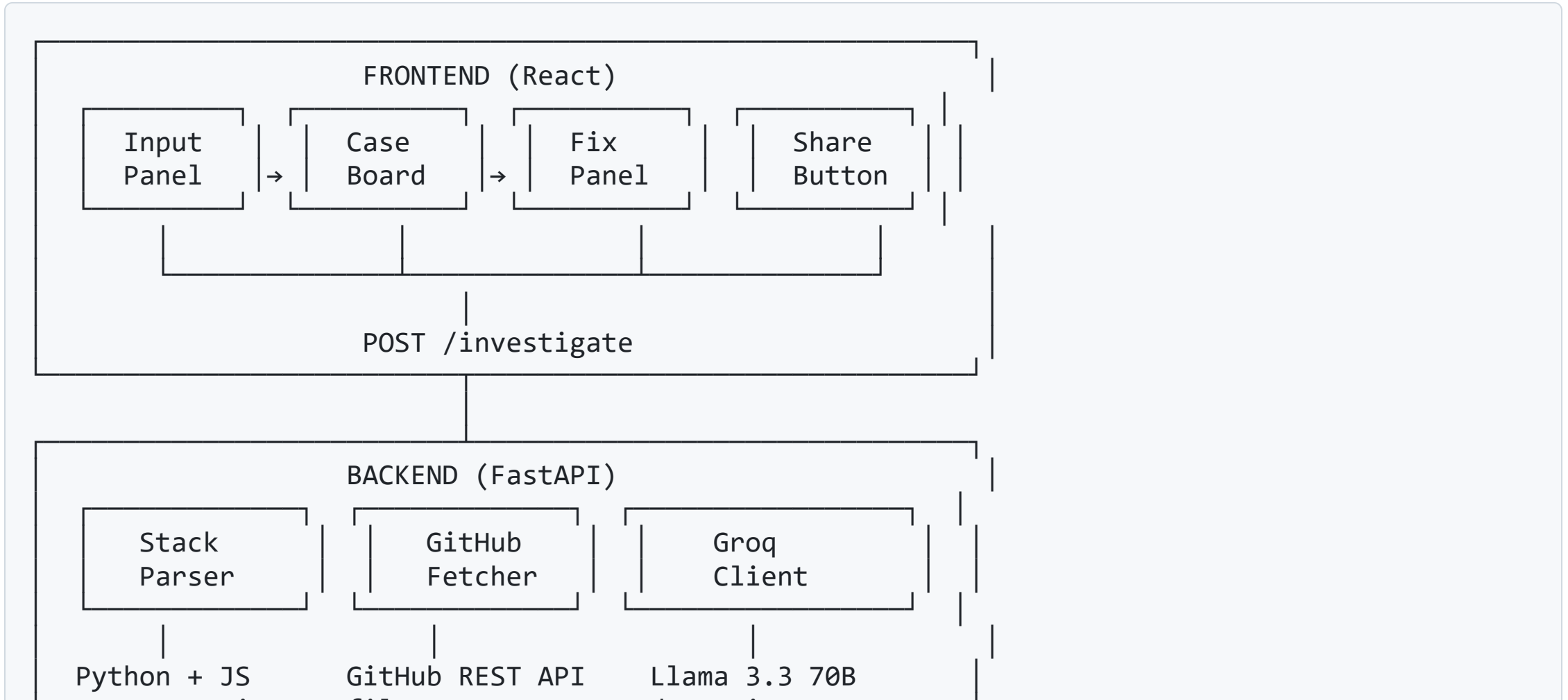
 Investigate

 Add Annotation

 Manage Files

Slide 6: Architecture

System Architecture



Slide 7: AI Integration

How We Use AI (Groq + Llama 3.3)

The Detective Prompt:

Our system prompt instructs the model to act as a "Senior Debugging Detective" with specific instructions:

1. **Read the stack trace** to understand what error occurred
2. **Examine each source file** carefully
3. **Trace the causal chain** — what happened BEFORE the error?
4. **Identify the TRUE root cause** (often different from where exception surfaced)
5. **Classify each location** by role (crime_scene, prime_suspect, accomplice, red_herring)
6. **Propose a concrete fix** with explanation

Slide 8: Real-World Utility

Who Benefits?

User	How Bug Whisperer Helps
Junior Developers	Understand complex crashes without senior help
Senior Engineers	Save hours debugging across large codebases
Open Source Maintainers	Triage bug reports faster
Hackathon Teams	Debug under time pressure
DevOps/SRE	Quick root cause analysis in production

Real Impact Example

Before Bug Whisperer:

- Developer sees `AttributeError: 'NoneType' object has no attribute 'strip'`

Slide 9: Phone-First Design

Mobile-First Execution

Bug Whisperer is designed **mobile-first** for the iQOO Hackathon:

1. **Camera Input** — Photograph stack traces from screens/papers
2. **PWA Installable** — Add to home screen like a native app
3. **Offline Support** — Core UI works without internet
4. **Web Share API** — Share investigations via native share sheet
5. **Haptic Feedback** — Vibration on success/error
6. **Touch Optimized** — 44px minimum touch targets
7. **Responsive Design** — Perfect on any screen size

Why This Fits the Hackathon

Phone-first execution — Camera input, PWA installability, offline UI

Slide 10: Future Vision

Post-Hackathon Roadmap

Phase 2: Production Features

- GitHub OAuth for private repos
- Follow imports/call graph beyond trace files
- Persist investigation history (Postgres/Supabase)
- CI/CD integration (auto-analyze failed builds)

Phase 3: Platform

- VS Code extension
- Slack/Discord bot
- REST API for teams
- Support for Java, Go, Ruby, Rust stack traces

Slide 11: Team & Ask

What We Need

1. **Shortlisting** — Get selected for the on-site Pune City Battle
2. **iQOO 15 Loaner** — Test mobile features on real hardware
3. **Mentorship** — Guidance on scaling the AI analysis

What We Bring

- Working prototype (backend + frontend)
- Unique UI concept (detective case board)
- Strong AI integration (not just wrapper)
- Mobile-first PWA ready for demo
- Real-world utility that judges will appreciate

Slide 12: Thank You

Bug Whisperer — AI Debugging Detective

"This isn't code review. It's investigation."

 Repository: github.com/your-team/bug-whisperer

 Contact: your-email@example.com

Built with ♥ for hackathon judges who know the pain of debugging at 2am